



**UNIVERSIDAD
NACIONAL AUTÓNOMA
DE MÉXICO**



**FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN**

Sistemas Distribuidos

S05 – Mexi-Banco

Alumno

Guerrero López Uriel Iván

NOMBRE DEL PROFESOR: Oscar Manuel Ruiz Hurtado

GRUPO: 02

SEMESTRE 2027-1

FECHA DE ENTREGA: Domingo 27 de septiembre de 2026

Desarrollo

Paso 1 · Clonado del repositorio

Primero cloné el repositorio correspondiente a la versión de la práctica:

```
uri82@MacBook-Air-de-Uriel Documents % git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Clonando en 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Recibiendo objetos: 100% (123/123), 46.71 KiB | 724.00 KiB/s, listo.
Resolviendo deltas: 100% (23/23), listo.
advertencia: |refs/tags/v05.1 cd46bb91a09070f8242c3b8acfb17caf2405cda no es un commit!
Nota: cambiando a '88203161aa31a371823715684a548d154e1516f8'.

Te encuentras en estado 'detached HEAD'. Puedes revisar por aquí, hacer
cambios experimentales y hacer commits, y puedes descartar cualquier
commit que hayas hecho en este estado sin impactar a tu rama realizando
otro checkout.

Si quieres crear una nueva rama para mantener los commits que has creado,
puedes hacerlo (ahora o después) usando -c con el comando checkout. Ejemplo:

git switch -c <nombre-de-nueva-rama>

O deshacer la operación con:

git switch -

Desactiva este aviso poniendo la variable de config advice.detachedHead en false
```

2. Levantar los servicios con Docker Compose

Después de clonar el proyecto intenté levantar los servicios utilizando:

```
docker compose up --build -d
```

En el primer intento Docker no pudo continuar porque Docker Desktop se encontraba pausado. Después de reanudar Docker Desktop volví a ejecutar el comando y la construcción de las imágenes se realizó correctamente.

Sin embargo, posteriormente apareció un problema con el puerto 8080. Docker indicó que el puerto ya estaba siendo utilizado mediante el mensaje:

```
Bind for 0.0.0.0:8080 failed: port is already allocated
```

Para encontrar qué contenedor estaba utilizando ese puerto ejecuté docker ps. Encontré que un contenedor de una práctica anterior llamado s04-web-1 estaba utilizando el puerto 8080. En lugar de eliminarlo, decidí solamente detenerlo mediante:

```
docker stop s04-web-1
```

Finalmente, tanto app como db aparecieron como healthy, por lo que pude continuar con las pruebas de la aplicación.

```
[*] up 3/3
  Image mexi-banco-app      Built      1.0s
  Container mexi-banco-db-1  Healthy   0.5s
  Container mexi-banco-app-1 Running    0.0s
ur182@MacBook-Air-de-Uriel mexi-banco % docker compose ps
NAME                IMAGE                                COMMAND                  SERVICE    CREATED   STATUS    PORTS
mexi-banco-app-1    sha256:42b2f69f8717f75c2fed9b4c264c6a8d8a5d0573ba0976c5fa6b76880d14bde7  "java -jar app.jar"     app        26 minutes ago    Up 26 minutes (healthy)    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine                 "docker-entrypoint.s..." db         26 minutes ago    Up 26 minutes (healthy)    5432/tcp
ur182@MacBook-Air-de-Uriel mexi-banco %
```

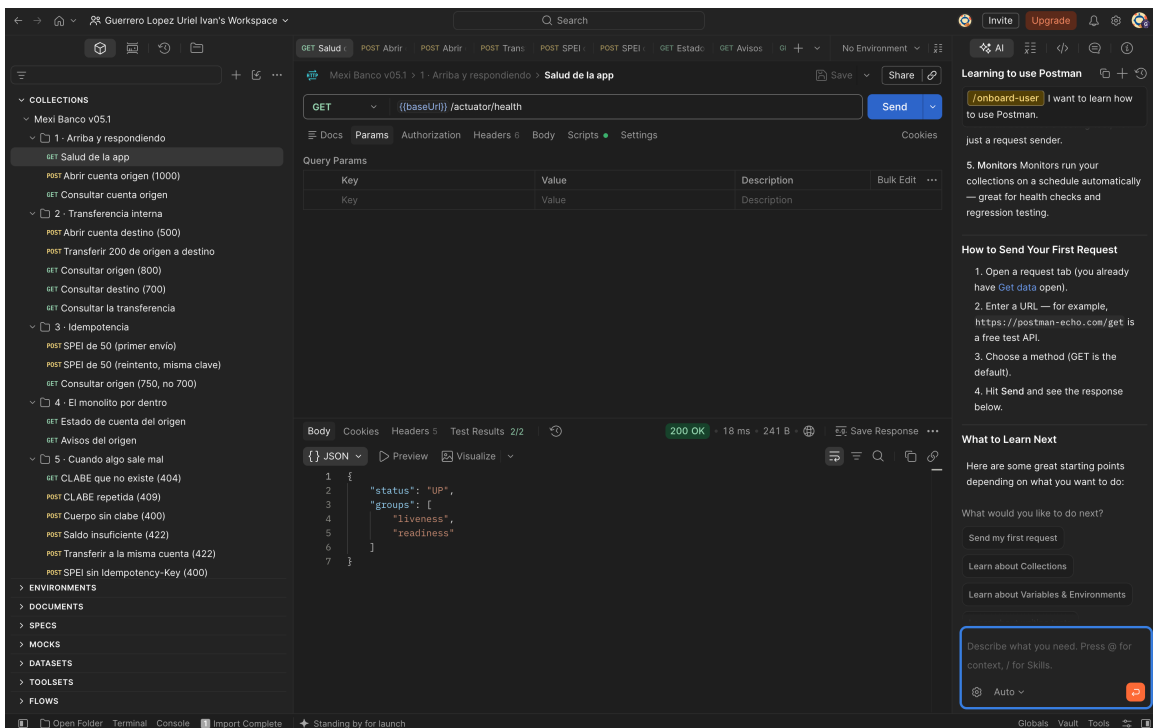
2. Probando los end-points

Una vez que los contenedores estuvieron funcionando correctamente, utilicé Postman para probar los endpoints de Mexi Banco.

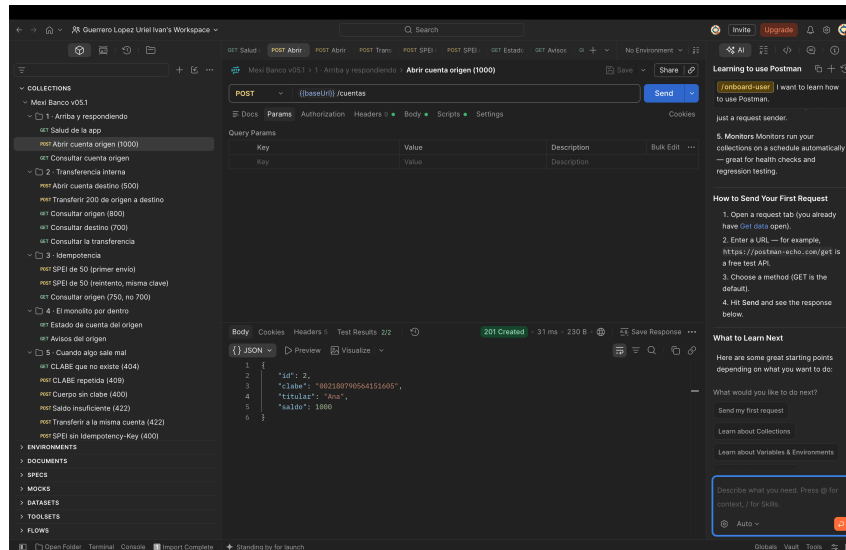
Primero realicé la consulta al endpoint de salud de la aplicación:

GET `{{baseUrl}}/actuator/health`

La petición respondió correctamente, lo que confirmó que la aplicación estaba disponible y que el contenedor app se encontraba funcionando.

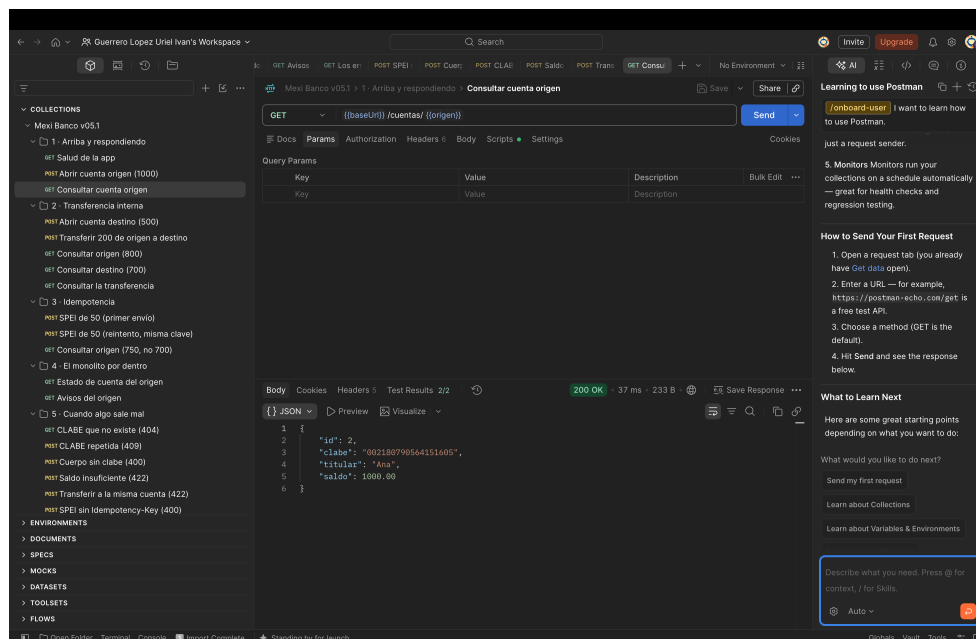


1.2. Abrir cuenta origen: `{{baseUrl}}/cuentas`

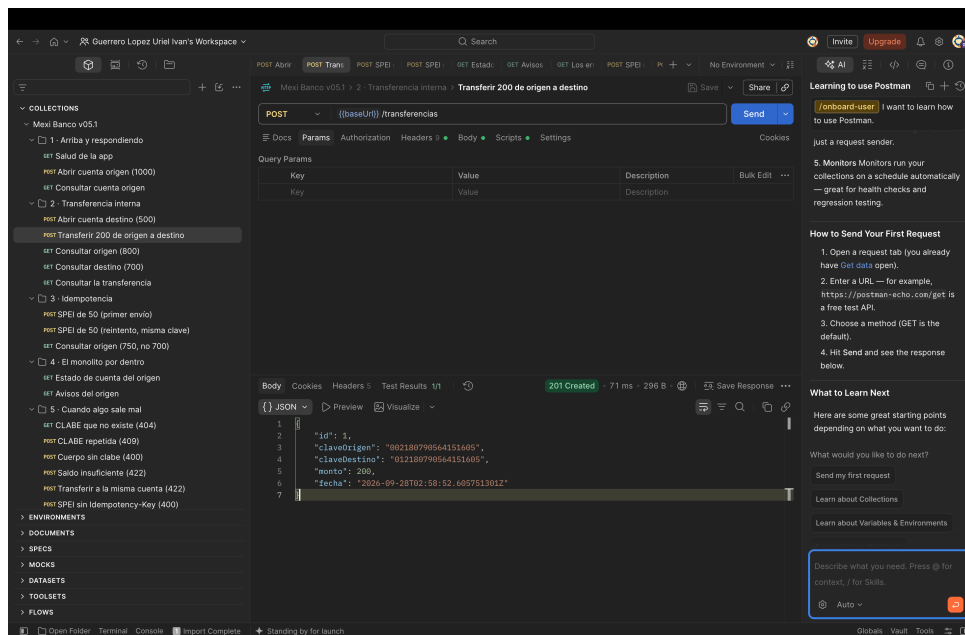


El endpoint de apertura de cuenta se utilizó para registrar una nueva cuenta bancaria dentro del sistema. En esta prueba se creó la cuenta de Ana, obteniendo una CLABE, un identificador y un saldo inicial de 1000. Esta operación permitió tener una cuenta sobre la cual realizar las siguientes pruebas de transferencias y movimientos.

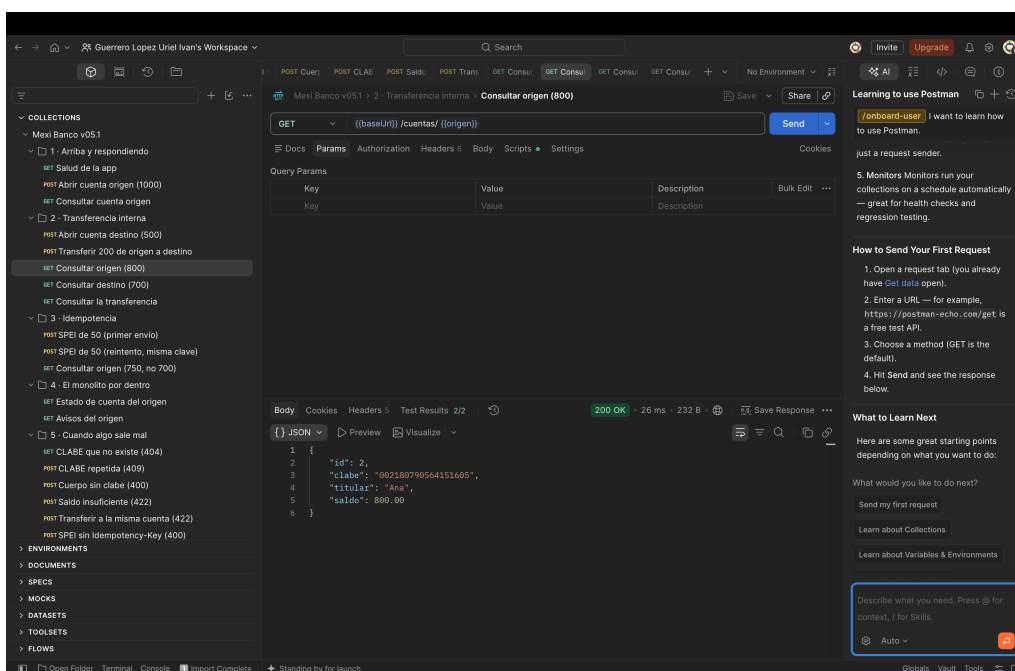
1.3. Consultar cuenta origen: {{baseUrl}}/cuentas/{{origen}}



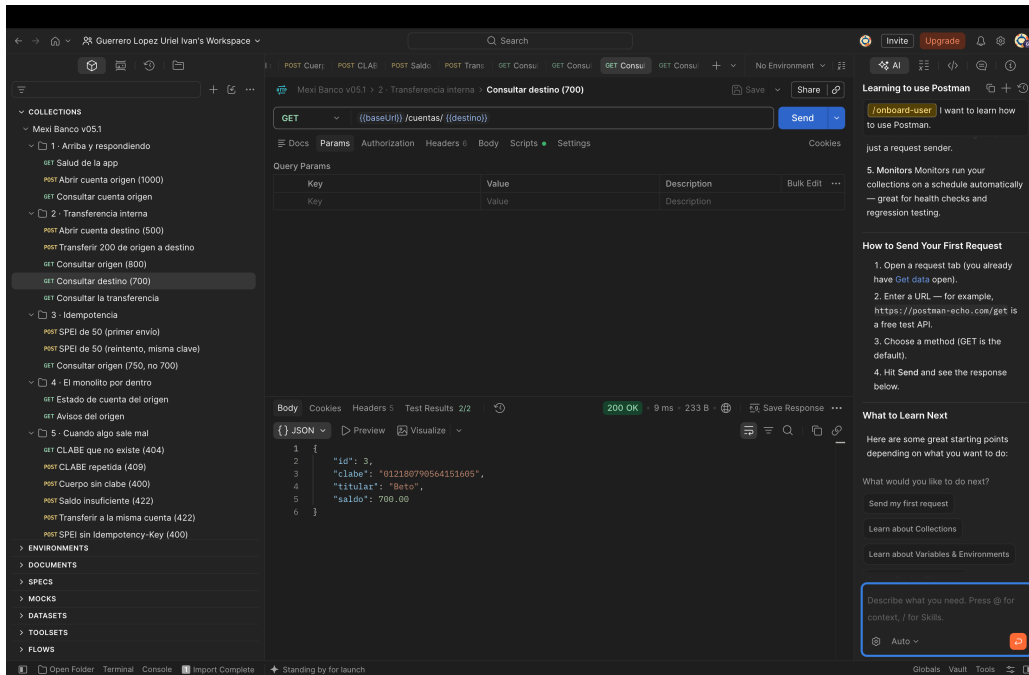
2. Transferencia interna



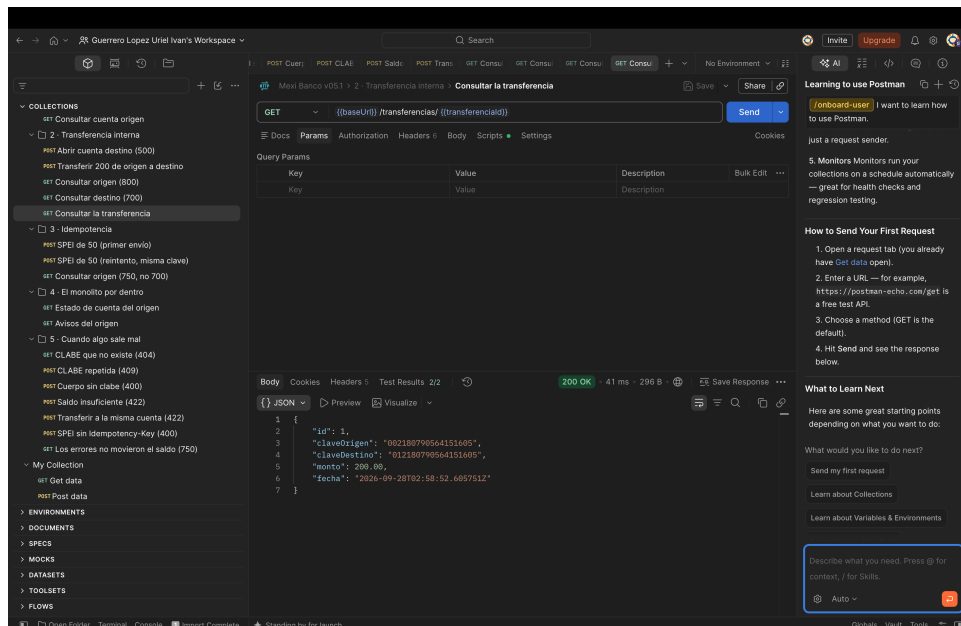
El endpoint de transferencia permite enviar dinero desde una cuenta origen hacia una cuenta destino. En esta prueba se realizó una transferencia de 200, por lo que el sistema registró el movimiento y actualizó el saldo de la cuenta origen de 1000 a 800.



Con la petición GET consulté los datos de la cuenta destino y pude comprobar que la cuenta del titular "Beto" muestra un saldo actualizado de 700.00. Esto permite verificar que la transferencia se realizó correctamente y que el saldo de la cuenta se actualizó.

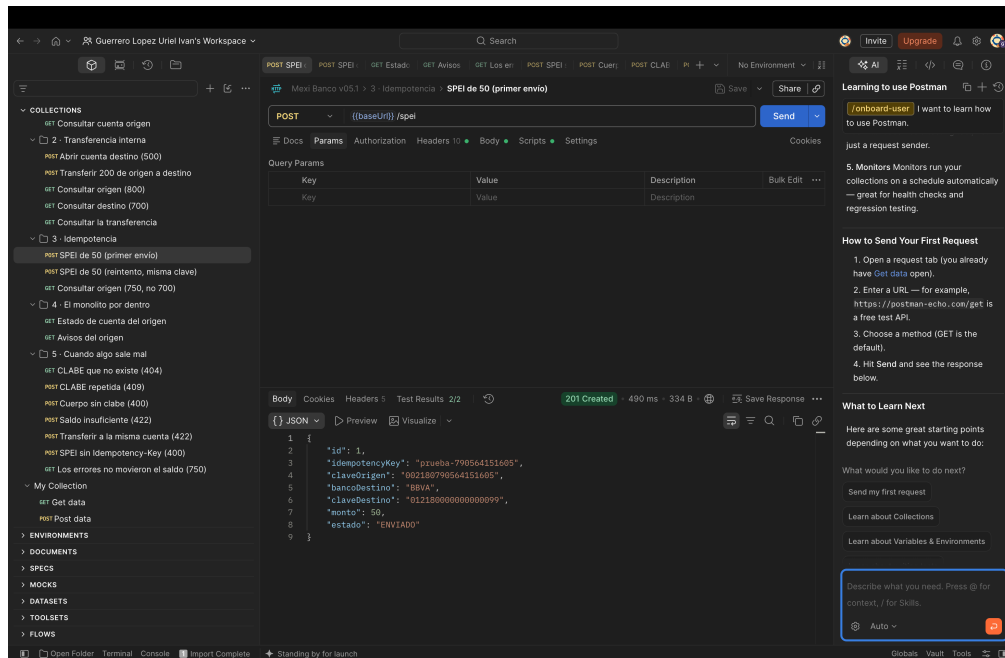


Con la petición GET consulté la transferencia realizada y pude revisar los datos específicos de la operación registrada con el identificador 1, como la cuenta de origen, la cuenta de destino, el monto y la fecha de la transferencia.

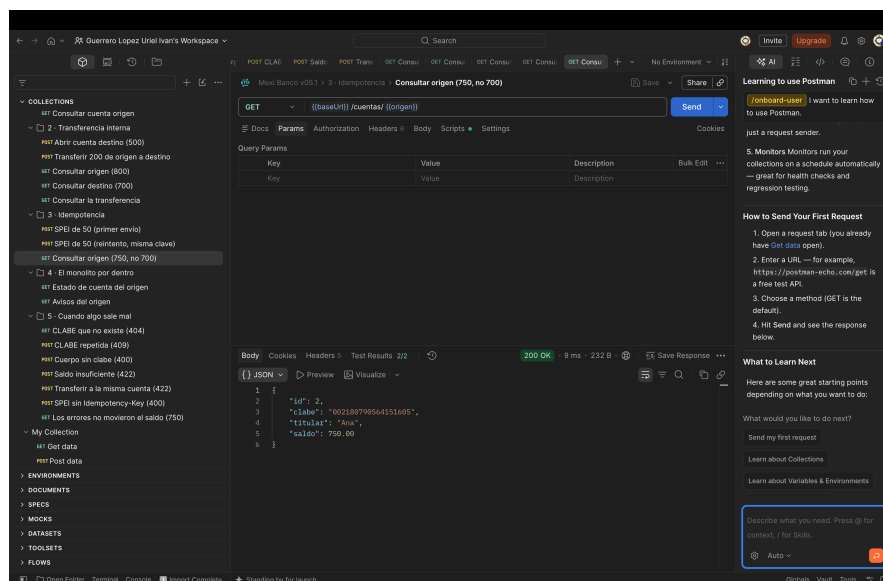


3. Idempotencia

3.1. SPEI de 50 (primer envío): {{baseUrl}}/spei



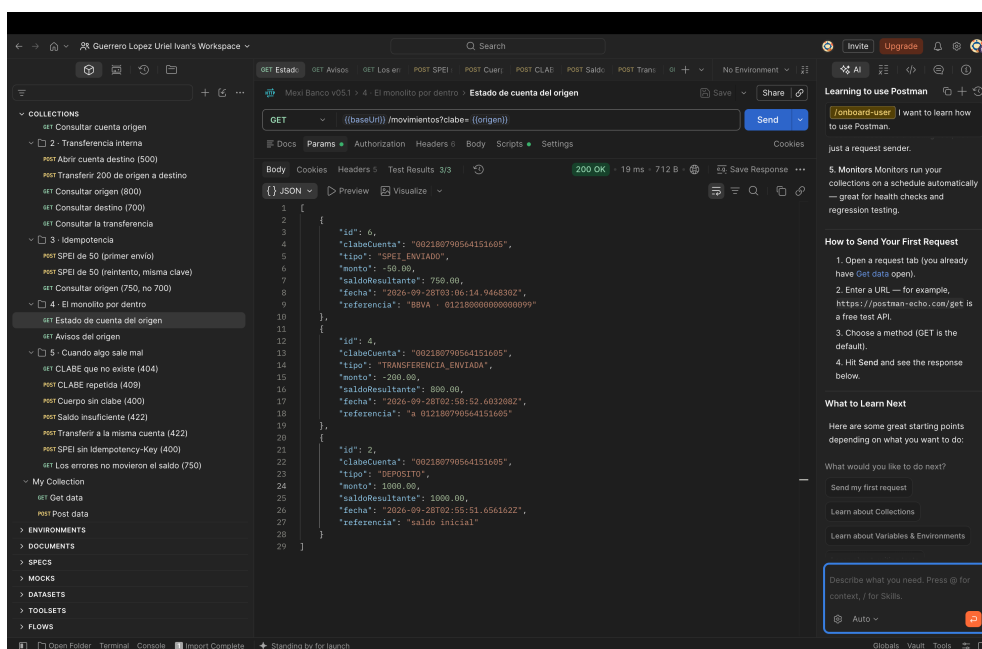
El endpoint de SPEI permite realizar una transferencia hacia una cuenta de otro banco. En esta prueba se realizó un envío de 50 hacia una cuenta de BBVA y se utilizó una clave de idempotencia para identificar la operación. La respuesta indicó que el SPEI fue procesado con el estado ENVIADO.



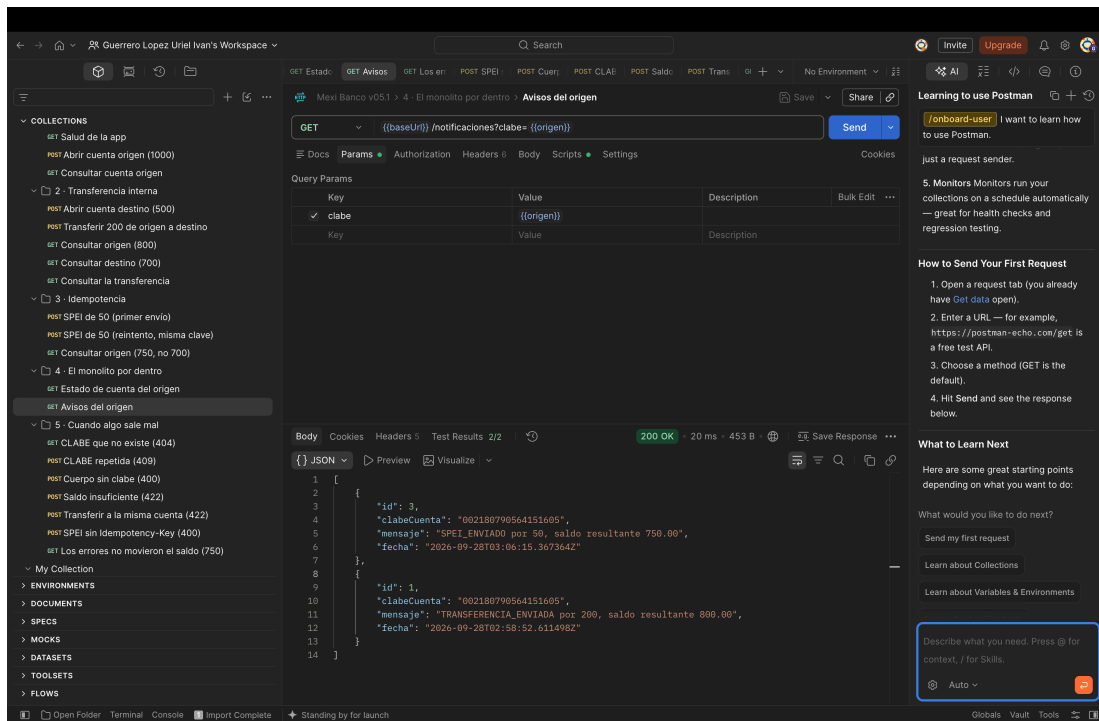
Con la petición POST realicé nuevamente la transferencia SPEI utilizando la misma clave de idempotencia. El sistema devolvió la misma información de la operación original, registrada con el identificador 1 y con estado "ENVIADO". Esto permitió comprobar que la operación no se procesa nuevamente cuando se utiliza la misma clave de idempotencia.

4. El monolito por dentro

4.1. Estado de cuenta del origen: `{{baseUrl}}/movimientos?clabe={{origen}}`



El endpoint de estado de cuenta permite consultar los movimientos realizados sobre una cuenta. En esta prueba se pudieron observar el depósito inicial de 1000, la transferencia de 200 y el SPEI de 50, junto con el saldo resultante después de cada operación. Al final, el saldo registrado fue de 750.

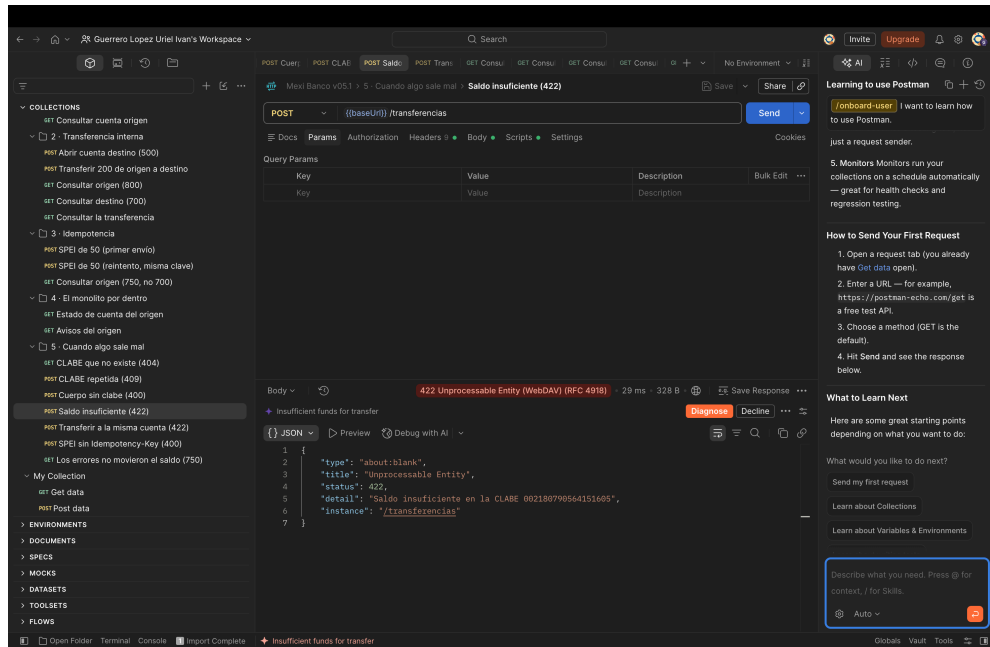


El endpoint de avisos permite consultar las notificaciones generadas a partir de las operaciones realizadas en la cuenta. En este caso se obtuvieron avisos relacionados con la transferencia y el envío SPEI, indicando el monto de cada operación y el saldo resultante.

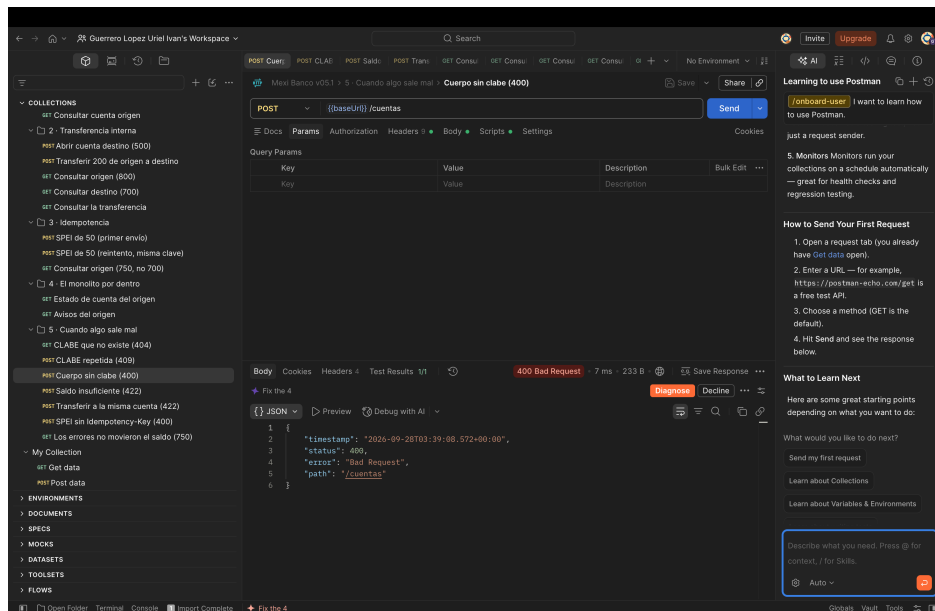
5. Cuando algo sale mal

Finalmente se probaron diferentes situaciones que deberían ser rechazadas por las reglas de negocio de la aplicación. Primero se intentó crear una cuenta utilizando una CLABE que ya existía, después se realizó una transferencia con saldo insuficiente y finalmente se intentó transferir dinero de una cuenta hacia ella misma. En los tres casos la aplicación rechazó correctamente las operaciones y devolvió códigos HTTP y mensajes que explican el motivo del error.

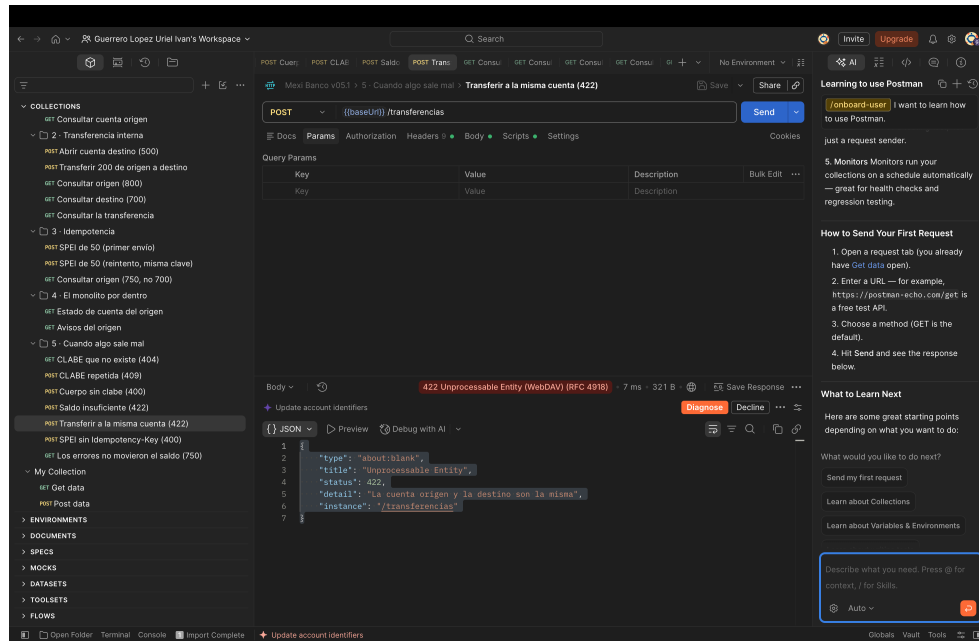
5.1. CLABE que no existe (404): `{{baseUri}}/cuentas/999999999999999999`



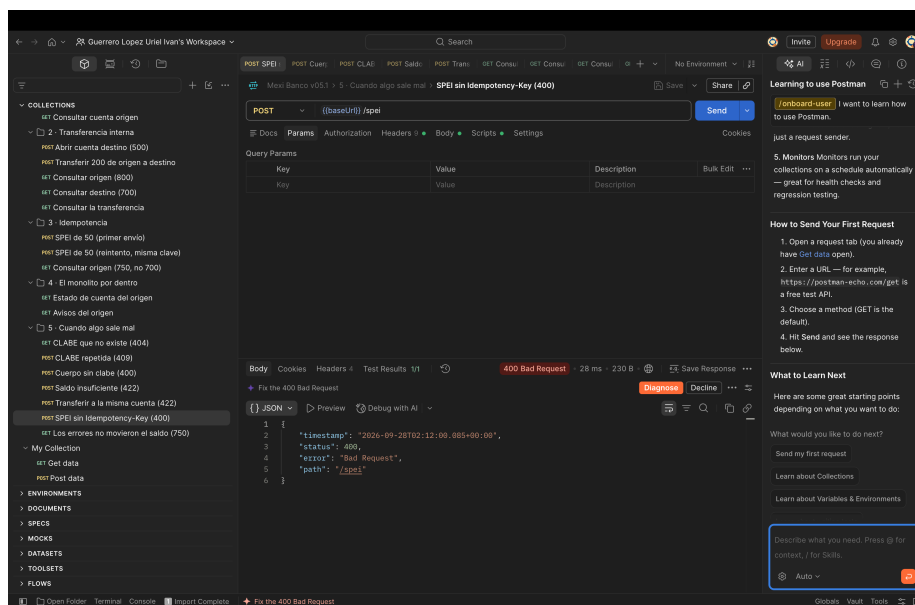
5.4. Cuerpo sin clave (400): {{baseUrl}}/cuentas



5.6. Transferir a la misma cuenta (422): {{baseUrl}}/transferencias



5.7. SPEI sin idempotency-Key (400): {{baseUrl}}/spei



Paso 4: Bájalo con y sin volumen

Después de realizar las pruebas de la API comprobé el comportamiento del volumen de PostgreSQL.

Primero ejecuté:

```
docker compose down
docker compose up -d
```

```
ur182@MacBook-Air-de-Uriel mexi-banco % docker compose down
[+] down 3/3
✓ Container mexi-banco-app-1 Removed 0.2s
✓ Container mexi-banco-db-1 Removed 0.1s
✓ Network mexibanco Removed 0.1s
ur182@MacBook-Air-de-Uriel mexi-banco % docker compose up -d
[+] up 3/3
✓ Network mexibanco Created 0.0s
✓ Container mexi-banco-db-1 Healthy 5.6s
✓ Container mexi-banco-app-1 Started 5.7s
ur182@MacBook-Air-de-Uriel mexi-banco %
```

En este caso los contenedores y la red fueron eliminados y posteriormente creados nuevamente, pero el volumen de PostgreSQL permaneció. Esto significa que los datos almacenados en la base de datos pueden mantenerse aunque los contenedores sean recreados.

Posteriormente realicé la segunda prueba utilizando:

```
docker compose down -v
docker compose up -d
```

```
ur182@MacBook-Air-de-Uriel mexi-banco % docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1 Removed 0.2s
✓ Container mexi-banco-db-1 Removed 0.1s
✓ Network mexibanco Removed 0.1s
✓ Volume mexi-banco_mexibanco_datos Removed 0.0s
ur182@MacBook-Air-de-Uriel mexi-banco % docker compose up -d
[+] up 4/4
✓ Network mexibanco Created 0.0s
✓ Volume mexi-banco_mexibanco_datos Created 0.0s
✓ Container mexi-banco-db-1 Healthy 5.7s
✓ Container mexi-banco-app-1 Started 5.7s
ur182@MacBook-Air-de-Uriel mexi-banco %
```

La diferencia fue que down -v también eliminó el volumen mexi-banco_mexibanco_datos. Al ejecutar nuevamente docker compose up -d, Docker tuvo que crear un volumen nuevo.

Con esta prueba pude comprobar que docker compose down y docker compose down -v tienen un comportamiento diferente respecto a la persistencia de los datos.

Paso 5 · Revisión del código

En este paso revisé la estructura del proyecto dentro de src/main/java/mx/mexibanco. La aplicación está organizada en diferentes módulos: cuenta, movimiento, transferencia, notificacion y spei. Cada módulo sigue una estructura de cuatro capas: controlador,

servicio, repositorio y entidad, donde cada una tiene una responsabilidad diferente dentro de la aplicación. Esta organización permite separar las funciones y facilita entender cómo está construido el monolito que posteriormente se irá dividiendo en diferentes servicios.

Paso 6 · Revisión del compose y Dockerfile

También revisé el archivo `compose.yaml` para entender cómo se levantan los diferentes componentes de la aplicación. En este caso se utilizan los servicios `app` y `db`, donde `app` corresponde a la aplicación y `db` a PostgreSQL. También revisé los `healthcheck`, que permiten comprobar que los servicios realmente estén funcionando, la red `mexibanco` que permite la comunicación entre los contenedores y el volumen utilizado para mantener los datos de la base de datos.

Por último, revisé el `Dockerfile`, que está dividido en dos etapas. La primera utiliza el JDK para descargar las dependencias y compilar la aplicación mediante Maven. La segunda utiliza el JRE y únicamente copia el archivo `.jar` generado para ejecutar la aplicación. De esta forma se separa el proceso de construcción del proceso de ejecución y se mantiene una imagen final más sencilla.